

Task & Content Analysis

Digital Literacy Fundamentals — Skill Hierarchy and Domain Classification

1. Purpose and Method

This document replaces an earlier flat skill list with a proper hierarchical task analysis, classified against Gagné's learning domains. The distinction matters instructionally, not just organizationally: a motor skill (dragging, tapping) requires modeling plus guided practice plus feedback. Verbal information (naming a taskbar) requires recognition and repetition. An intellectual skill (discriminating a single click from a double-click by result) requires practice against varied examples, not just exposure. Treating all skills the same way — as a flat list of things to “cover” — is not defensible instructional design. Each skill below is tagged by domain, and each domain implies a different teaching strategy, carried forward into the Instructional Strategy & Approach document.

The hierarchy below also replaces an informal “prerequisite knowledge” column with an actual dependency structure: which skills must be mastered before which others can be attempted at all. This is what drives lesson order in the Content Outline/Blueprint — not topic grouping.

2. Domain-Classified Skill Inventory

Domain key: MS = Motor Skill, VI = Verbal Information, IS = Intellectual Skill (discrimination/rule use).

Lesson 2 — The Mouse

Skill	Domain	Instructional Implication
Recognize that a single tap/click selects or activates an item	MS + VI	Needs modeling (show it happening) plus one guided attempt with immediate confirmation — not just a definition.
Distinguish single-click from double-click by result, not by count	IS	Needs discrimination practice: show both outcomes side by side so the learner learns to predict result, not just perform a count.
Recognize right-click as “more options,” not an error state	VI + IS	Needs explicit naming (verbal information) plus a rule (“this menu means more choices, not a mistake”) reinforced through a low-stakes example.
Recognize drag as a single continuous action (press-hold-move-release), not four separate steps	MS	Needs demonstration via guided step-through rather than a freeform attempt, per the input-model decision below.

Lesson 3 — The Keyboard

Skill	Domain	Instructional Implication
Locate and use letter/number keys to produce visible text	MS	Needs hands-on repetition in a low-stakes text field with immediate visible feedback.
Use Backspace/Delete to correct an entry	MS + IS	Needs a deliberately-seeded “mistake” the learner corrects, not just a definition of the key's function.

Skill	Domain	Instructional Implication
Use Enter/Return to submit or confirm	VI	Needs explicit naming — learners commonly search for an on-screen button instead, so the concept has to be told, not just demonstrated.
Use Shift to produce a capital letter	MS + IS	Needs guided practice pairing the hold-and-press sequence with a visible before/after (lowercase vs. capital).

Lesson 4 — Windows

Skill	Domain	Instructional Implication
Recognize a window as a distinct, bounded space, separate from the desktop behind it	IS	Conceptual, not motor — needs a visual comparison (window open vs. closed) more than a hands-on rep.
Open and close a window using a visible control	MS	Needs a guided first attempt with a clearly visible close control, not a hidden or gestural one.
Minimize and restore a window	MS + IS	Needs the learner to see the same window persist in the taskbar after minimizing, correcting the common misconception that minimizing deletes it.
Switch between two open windows	MS + IS	Depends on the minimize/restore concept above — sequenced after it, not before.

Lesson 5 — The Desktop

Skill	Domain	Instructional Implication
Recognize desktop icons as launch points, distinguishable from decoration	VI + IS	Needs explicit naming plus a rule for telling icons apart from background imagery.
Recognize the taskbar/dock as a persistent home base separate from any open window	VI + IS	Depends on the window concept from Lesson 4 — the taskbar's value only makes sense once the learner has experienced losing track of a window.

3. Prerequisite Hierarchy

This replaces the earlier flat table's informal ordering with an explicit dependency chain. An arrow means “must be introduced before.” This hierarchy is the actual basis for lesson and screen order in the Storyboard and Content Outline/Blueprint — not topic convenience.

- Single click → double-click discrimination → right-click recognition → drag (guided)
- Typing letters/numbers → Backspace correction → Enter to confirm → Shift for capitals
- Window-as-space concept → open/close → minimize/restore → switch between windows
- Desktop icons as launch points → (requires: single click) → taskbar as home base → (requires: minimize/restore, from Windows)

Two cross-domain dependencies fall out of this chain and directly determine course-level lesson order: the Desktop lesson's taskbar concept depends on the Windows lesson's minimize/restore concept, and every domain depends on the single-click concept taught first in the Mouse lesson. This is why Mouse comes first and Desktop comes last among the four skill lessons — not because of topic logic, but because of this dependency chain.

4. Input Model Decision

Locked for the entire course, applies to every interactive screen in the capstone and to any hands-on moment inside Lessons 2–5:

- Learner interacts by tapping directly on icons, windows, and controls — the same touch pattern used everywhere else in the LMS. There is no draggable on-screen cursor for the learner to control.
- Each interactive element is paired with a plain-language annotation naming the real-laptop equivalent (e.g., a tap on an icon is labeled “on a real laptop, this is a mouse click”).
- Drag-dependent skills (window move, drag-to-select) are demonstrated via a short guided step-through rather than requiring a freeform drag gesture.

Why

Per Cognitive Load Theory: simulating a draggable cursor stacks a second, unfamiliar motor skill — controlling a proxy cursor via finger movement — on top of the actual concept being taught. That's extraneous load competing with germane load for the same limited working memory, and it introduces a failure mode this population doesn't need: a learner who can't reliably drag a small on-screen cursor fails the interaction, not the concept, which conflicts with the trauma-informed principle of low-stakes, unambiguous error recovery already established in the Needs & Audience Analysis.

Trade-off, stated plainly: this model is less literally realistic than a simulated cursor would be. If future learner feedback indicates learners are getting stuck on cursor manipulation specifically — not general laptop unfamiliarity — that would be a reason to revisit this decision in a later revision, not evidence it was wrong at the time it was made.

5. Simulated Environment — Build Specification (Capstone Lesson Only)

This section specifies what the capstone's simulated environment has to render and respond to. It lives here, not in the Style Guide, because it's build-spec information: the Style Guide governs how things look, this governs what the environment has to be able to do.

Component	Required Behavior	Explicitly Out of Scope
Desktop surface	Renders a static background with a fixed set of tappable icons, labeled in plain text beneath each icon.	Customizable wallpaper, rearrangeable icons, right-click context menus on the desktop itself
Icons	Tap opens an associated window. Visual state change on tap (brief highlight) confirms the tap registered.	Double-click-to-open behavior — single tap is sufficient, consistent with the input model above
Windows	Open, close (X control), minimize (– control), and restore. At least two windows can be open at once and switched between via a taskbar entry.	Resizable/draggable window chrome, overlapping z-index management beyond simple front/back

Component	Required Behavior	Explicitly Out of Scope
Taskbar	Persistent strip showing open windows as tappable entries; tapping switches focus to that window.	System tray icons, clock, notifications — anything not tied to a taught skill
Keyboard input	A tappable on-screen keyboard (or the device's native keyboard) that writes into a visible text field, supporting letters, numbers, Backspace, Enter, and Shift.	Autocomplete, spellcheck, special characters/symbols beyond what a skill requires
Mouse-equivalent actions	Single tap = click. Press-and-hold-then-move (where used) = drag, demonstrated via guided step-through, not freeform.	A persistent on-screen cursor graphic that tracks continuous finger movement
Progress/state	Tracks which capstone tasks are complete, consistent with the platform-wide no-visible-attempt-tracking and no-comparative-element rules established in the Accessibility & Inclusivity Checklist.	Timing data, error/attempt counts visible to the learner or facility staff

6. Content Dependency Analysis

- The entire course, and especially the capstone, depends on the simulated environment functioning correctly on the target tablet hardware. Facility-to-facility hardware variance (screen size, touch sensitivity, per the Needs & Audience Analysis on file) means this course has a higher build-fidelity bar than a text-and-video module, because the interaction itself, not just the content, has to hold up across that variance.
- No lesson in this course depends on outbound links, external sites, or any tool outside the platform — built platform-independent from the start, consistent with the Access & Completion standard already established elsewhere in this portfolio.

7. Findings

- Fourteen skills across four domains are enough to fill four full lessons without introducing files, folders, or email — confirming that scope cut (deferred to a later course) was reasonable.
- The prerequisite hierarchy in Section 3, not topic convenience, is what fixes lesson order: Mouse first (everything depends on single-click), Desktop last (taskbar depends on the Windows lesson's minimize/restore concept).
- The input-model decision in Section 4 is the single choice every downstream document depends on — Storyboard, Style Guide, and Claude Design prompts all need to reflect tap-based interaction with real-laptop-equivalent labeling, not a simulated cursor.